Cheatsheet — CSS

> **Materia:** Programación Orientada a la Web
> **Uso:** repaso rápido y consulta durante el parcial.
> **Base:** CSS 2.1 + módulos de CSS3 (selectores modernos, tipografía, box model, layout y responsive).

Índice

1. Sintaxis y formas de inclusión
2. Selectores
3. Combinadores
4. Pseudo-clases
5. Pseudo-elementos
6. Cascada, herencia y especificidad
7. Variables, `@layer` y `@import`
8. Unidades y colores
9. Tipografía
10. Propiedades de texto
11. Modelo de caja
12. Fondos
13. `display`, `position`, `float` y `z-index`
14. Flexbox
15. Grid
16. Responsive: media queries
17. Transiciones, transformaciones y animaciones
18. Extras de CSS3
19. Errores típicos y checklist
20. Plantilla completa

1. Sintaxis y formas de inclusión

Regla CSS

```
selector {
  propiedad: valor;
  otra-propiedad: valor;
}
```

```
h1 {
  color: #0f4c81;
  font-size: 24px;
  text-align: center;
}
```

- **Selector:** a qué elementos se aplica.
- **Declaración:** par `propiedad: valor`.
- Cada declaración termina en `;`.
- **Comentarios:** `/* ... */` (pueden ocupar varias líneas; no se anidan).

Las tres formas de incluir CSS

1. En línea (inline) — sobre un elemento:

```
<p style="color: red; font-weight: bold;">Texto rojo</p>
```

2. Interna (en el `head`) — para un solo documento:

```
<head>
  <style>
    h1 { color: red; font-style: italic; }
  </style>
</head>
```

3. Externa (enlazada) — la forma recomendada:

```
<link rel="stylesheet" href="css/estilos.css" type="text/css" media="screen">
```

4. Importada — desde otra hoja o dentro de `<style>`:

```
@import url("base.css");
@import url("print.css") print;
```

Forma	Alcance	Recomendación
Inline	Un elemento	Evitar; difícil de mantener.
Interna	Un documento	Casos puntuales.
Externa	Varios documentos	**Preferida.**
`@import`	Varias hojas	Útil para organizar; puede demorar la carga.

Atributo `media`

Valor	Medio
`screen`	Pantalla (por defecto).
`print`	Impresora.
`projection`	Proyector.
`aural`	Sintetizador de voz.
`braille`	Dispositivos braille.
`tty`	Terminal de caracteres.
`tv`	Televisión.
`all`	Todos.

```
<link rel="stylesheet" href="impresion.css" media="print">
```

Estilos persistentes, preferidos y alternos

```
<!-- Persistente: sin title -->
<link rel="stylesheet" href="base.css" type="text/css" media="screen">

<!-- Preferido: con title -->
<link rel="stylesheet" href="color.css" type="text/css" title="Color">

<!-- Alterno -->
<link rel="alternate stylesheet" href="alto-contraste.css" title="Alto contraste">
```

---

2. Selectores

Selector	Sintaxis	Selecciona
Universal	`*`	Todos los elementos.
Tipo	`p`	Todos los `<p>`.
Clase	`.destacado`	Elementos con `class="destacado"`.
Clase + tipo	`p.destacado`	`<p>` con esa clase.
ID	`#principal`	El elemento con `id="principal"`.
Atributo	`input[type="email"]`	Elementos con ese atributo/valor.
Atributo parcial	`a[href^="https"]`	`href` que **empieza** con `https`.
Agrupación	`h1, h2, h3`	Todos los de la lista.

```
* { box-sizing: border-box; }

p { line-height: 1.5; }

.destacado { background: #fff3cd; }

p.destacado { font-weight: bold; }
```

```
#principal { max-width: 960px; margin: 0 auto; }

input[type="email"] { border-color: #0f4c81; }

a[href^="https"]::after { content: " ↗"; }
a[href$=".pdf"]::after { content: " (PDF)"; }
a[href*="w3.org"] { color: green; }
```

Operador de atributo	Significado
`[attr]`	Tiene el atributo.
`[attr="x"]`	Valor exacto.
`[attr~="x"]`	Contiene la palabra `x`.
`[attr|="x"]`	Empieza con `x` o `x-`.
`[attr^="x"]`	Empieza con `x`.
`[attr$="x"]`	Termina con `x`.
`[attr*="x"]`	Contiene `x`.

> **Clases vs ID:** la clase se reutiliza y agrupa elementos con el mismo estilo; el `id` es **único** en el documento y sirve también para anclas y JavaScript.

3. Combinadores

Combinador	Sintaxis	Significado
Descendiente	`article p`	Todo `<p>` dentro de `<article>` (cualquier nivel).
Hijo directo	`article > p`	Solo hijos directos.
Hermano adyacente	`h2 + p`	El `<p>` inmediatamente después de un `<h2>`.
Hermanos posteriores	`h2 ~ p`	Todos los `<p>` hermanos que siguen a un `<h2>`.
Agrupación	`h1, h2`	Ambos selectores.

```
nav a { color: white; }          /* descendientes */
nav > ul { list-style: none; }   /* hijos directos */
h2 + p { margin-top: 0; }        /* hermano siguiente */
h2 ~ p { color: #475569; }       /* todos los siguientes */
```

> `h3 em` **no** es lo mismo que `h3, em`: el primero aplica solo a `<em>` dentro de `<h3>`; el segundo, a todos los `<h3>` y a todos los `<em>`.

4. Pseudo-clases

Sintaxis: `selector:pseudo-clase { ... }`

Enlaces y acción

Pseudo-clase	Cuándo se aplica
`:link`	Enlace no visitado.
`:visited`	Enlace visitado.
`:hover`	Mouse encima.
`:active`	Mientras se hace clic.
`:focus`	Tiene el foco (teclado o clic).
`:focus-visible`	Foco visible por teclado.

```
a:link    { color: #0f4c81; }
a:visited { color: #6b21a8; }
a:hover   { text-decoration: underline; }
a:active  { color: #dc2626; }
```

```
/* Orden recomendado: LVHA */
/* :link → :visited → :hover → :active */
```

Formularios y estado

Pseudo-clase	Significado
`:checked`	Checkbox/radio marcado.
`:disabled` / `:enabled`	Deshabilitado / habilitado.
`:required` / `:optional`	Con o sin `required`.
`:valid` / `:invalid`	Validación del navegador.
`:in-range` / `:out-of-range`	Dentro/fuera de `min`-`max`.
`:read-only` / `:read-write`	Solo lectura / editable.
`:placeholder-shown`	Se muestra el placeholder.

```
input:required { border-left: 3px solid #dc2626; }
input:valid    { border-left: 3px solid #16a34a; }
input:invalid  { background: #fef2f2; }
input:focus    { outline: 2px solid #0f4c81; }
```

Estructurales

Pseudo-clase	Significado
`:first-child`	Primer hijo de su padre.
`:last-child`	Último hijo.
`:only-child`	Único hijo.
`:nth-child(n)`	n-ésimo hijo.
`:nth-last-child(n)`	n-ésimo desde el final.
`:first-of-type`	Primero de su tipo.
`:last-of-type`	Último de su tipo.
`:nth-of-type(n)`	n-ésimo de su tipo.
`:only-of-type`	Único de su tipo.
`:empty`	Sin hijos ni texto.
`:root`	Elemento raíz (`<html>`).
`:target`	Elemento apuntado por la URL (`#id`).
`:not(selector)`	Todo lo que **no** coincide.

```
li:nth-child(odd)  { background: #f1f5f9; }  /* impares */
li:nth-child(even) { background: #ffffff; }  /* pares */
li:nth-child(3n+1) { font-weight: bold; }    /* 1, 4, 7, ... */
p:not(.nota)       { margin-bottom: 1rem; }
```

Lógicas modernas

```
:is(h1, h2, h3) { text-wrap: balance; }

:where(nav, footer) a { color: inherit; }

.card:has(img) { grid-template-columns: 8rem 1fr; }

form:has(input[type="checkbox"]:checked) {
  border: 2px solid green;
}
```

Selector	Qué hace	Especificidad
`:is(a, b)`	Agrupa selectores.	La del más específico del grupo.
`:where(a, b)`	Agrupa selectores.	**Siempre 0.**
`:has(sel)`	Selecciona el padre según su contenido.	La del selector interno.

> `:where()` es ideal para resets y estilos base porque no "gana" ninguna guerra de especificidad. `:has()` permite estilar un contenedor según lo que tiene adentro.

5. Pseudo-elementos

Sintaxis: `selector::pseudo-elemento { ... }`

Pseudo-elemento	Qué hace
`::before`	Inserta contenido antes del elemento.
`::after`	Inserta contenido después.
`::first-line`	Primera línea del texto.
`::first-letter`	Primera letra.
`::selection`	Porción seleccionada por el usuario.
`::placeholder`	Texto del placeholder.
`::marker`	Viñeta o número de un ítem de lista.

```
h1::before {
  content: "★ ";
  color: #f59e0b;
}

a.externo::after {
  content: " ↗";
}

p::first-letter {
  font-size: 3em;
  float: left;
  line-height: 1;
}

::selection {
  background: #fde68a;
  color: #111827;
}
```

> `::before` y `::after` **requieren** la propiedad `content`, incluso si es `content: ""`. No se debe poner información indispensable solo ahí: el contenido esencial va en el HTML.

6. Cascada, herencia y especificidad

La cascada

Cuando varias reglas afectan al mismo elemento, el navegador decide en este orden:

1. **Origen e importancia** (agente de usuario, usuario, autor; `!important` invierte parte del orden).
2. **Capas** (`@layer`).
3. **Especificidad** del selector.
4. **Orden de aparición** (si empatan, gana la última regla).

Orígenes:

Origen	Ejemplo
Agente de usuario	Estilos por defecto del navegador.
Usuario	Preferencias o extensiones.
Autor	La hoja de estilos del sitio.

Herencia

- **Se heredan:** color, `font-family`, `font-size`, `line-height`, `text-align`, `visibility`, etc.
- **No se heredan:** `margin`, `padding`, `border`, `background`, `width`, `height`, `display`, `position`.

```
body {
  color: #1f2937;
```

```
  font-family: system-ui, sans-serif;
}

/* .card hereda color y tipografía, pero no el borde */
.card { border: 1px solid #cbd5e1; }
```

Valores de control: `inherit` (hereda del padre), `initial` (valor inicial), `unset` (hereda si es heredable, si no `initial`), `revert` (vuelve al estilo del origen anterior).

Especificidad

Se calcula por columnas `(ID, clases/atributos/pseudo-clases, tipos/pseudo-elementos)`:

Selector	Especificidad
`*`	`0,0,0`
`p`	`0,0,1`
`.card`	`0,1,0`
`p.card`	`0,1,1`
`#main`	`1,0,0`
`#main .card p`	`1,1,1`
`style="..."` (inline)	Gana a las hojas (sin `!important`).
`!important`	Se impone sobre lo anterior.

```
/* De menor a mayor peso */
article p        { color: #334155; }  /* 0,0,2 */
.card p          { color: #0f766e; }  /* 0,1,1 */
.card .titulo    { color: #b91c1c; }  /* 0,2,0 */
#catalogo .titulo{ color: #7c3aed; }  /* 1,1,0 */
```

> La especificidad **no es una suma decimal**: `0,1,0` le gana a `0,0,10`. Se compara columna por columna.

`!important`

```
.alerta { color: red !important; }
```

- Se usa como **excepción**, no como solución habitual.
- Dificulta el mantenimiento y las sobrescrituras.
- Antes de usarlo, revisar capas, origen y especificidad.

`@layer`

```
@layer reset, base, components, utilities;

@layer base {
  body { font-family: system-ui, sans-serif; }
}

@layer components {
  .button { background: #0f4c81; color: white; }
}
```

- El orden se declara al principio.
- En reglas normales, la capa **posterior** gana.
- Los estilos **sin capa** superan a los normales en capas.

---

7. Variables, `@layer` y `@import`

Variables CSS (custom properties)

```
:root {
  --color-primario: #0f4c81;
  --color-texto: #1f2937;
  --radio: 8px;
  --espacio: 1rem;
}

.button {
```

```
  background: var(--color-primario);
  color: white;
  border-radius: var(--radio);
  padding: var(--espacio);
}

/* Valor de respaldo */
.titulo { color: var(--color-titulo, #111827); }
```

- Se declaran con `--nombre` y se usan con `var(--nombre)`.
- Se heredan y se pueden redefinir por contexto (tema claro/oscuro).
- Son la base de los *design tokens*.

Organización sugerida

```
@import url("reset.css");
@import url("base.css");
@import url("layout.css");
@import url("componentes.css");
@import url("utilidades.css");
```

---

8. Unidades y colores

Unidades de longitud

Unidad	Tipo	Referencia
`px`	Relativa (práctica)	Píxel del dispositivo.
`em`	Relativa	Tamaño de fuente del elemento.
`rem`	Relativa	Tamaño de fuente de la raíz (`html`).
`%`	Relativa	Depende de la propiedad.
`vw` / `vh`	Relativa	1 % del ancho / alto del viewport.
`vmin` / `vmax`	Relativa	El menor / mayor de `vw` y `vh` .
`ch` / `ex`	Relativa	Ancho de “0” / altura de “x”.
`cm` , `mm` , `in` , `pt` , `pc`	Absoluta	Medidas físicas (impresión).

```
html { font-size: 16px; }      /* 1rem = 16px */
h1   { font-size: 2rem; }      /* 32px */
p    { font-size: 1.125rem; }  /* 18px */
.hero { min-height: 60vh; }
```

> `0` no necesita unidad: `margin: 0;` es válido. `1.3 em` **no** es válido; `1.3em` sí.

Colores

Formato	Ejemplo
Nombre	`red` , `blue` , `teal`
Hexadecimal	`#ff0000` , `#f00`
RGB	`rgb(255, 0, 0)`
RGBA	`rgba(255, 0, 0, 0.5)`
HSL	`hsl(0, 100%, 50%)`
HSLA	`hsla(0, 100%, 50%, 0.5)`

```
:root {
  --primario: #0f4c81;
  --texto: rgb(31, 41, 55);
  --fondo: hsl(210, 40%, 98%);
  --sombra: rgba(0, 0, 0, 0.15);
}
```

Transparencia: `opacity: 0.5` afecta al elemento completo (incluidos hijos); `rgba()` / `hsla()` solo al color.

---

9. Tipografía

```
body {
  font-family: "Segoe UI", Arial, Helvetica, sans-serif;
  font-size: 16px;
  font-weight: 400;
  font-style: normal;
  line-height: 1.5;
}

h1 {
  font-family: Georgia, "Times New Roman", serif;
  font-size: 2.5rem;
  font-weight: 700;
  letter-spacing: -0.02em;
}

.codigo {
  font-family: Consolas, "Courier New", monospace;
}
```

Propiedad	Valores principales
`font-family`	Lista de fuentes + familia genérica de respaldo.
`font-size`	`px`, `rem`, `em`, `%`, `small`, `large`, etc.
`font-weight`	`normal` (400), `bold` (700), `100`–`900`, `bolder`, `lighter`.
`font-style`	`normal`, `italic`, `oblique`.
`font-variant`	`normal`, `small-caps`.
`line-height`	Número, longitud o porcentaje.
`letter-spacing`	Espacio entre letras.
`word-spacing`	Espacio entre palabras.

Familias genéricas

Genérica	Ejemplo
`serif`	Times, Georgia.
`sans-serif`	Arial, Helvetica, Segoe UI.
`monospace`	Courier, Consolas.
`cursive`	Zapf-Chancery.
`fantasy`	Western.

Shorthand `font`

```
p { font: italic bold 12pt/1.5 "Times New Roman", serif; }
/* estilo  peso  tamaño/line-height  familia */
```

> El shorthand `font` **resetea** las propiedades no indicadas a su valor inicial. Cuidado al usarlo.

`@font-face` (Web Fonts)

```
@font-face {
  font-family: "Museo Sans";
  src: url("/fonts/museo-sans.woff2") format("woff2");
  font-weight: 400;
  font-style: normal;
  font-display: swap;
}

body {
  font-family: "Museo Sans", Arial, sans-serif;
}
```

- Cada peso/estilo necesita su propia declaración (`400`, `700`, etc.).
- `font-display: swap` muestra el texto con la fuente de respaldo mientras carga.
- Formatos: **WOFF2/WOFF** (recomendados), TTF, EOT (IE antiguo), SVG (obsoleto).

- Fuentes gratuitas: Google Fonts, Font Squirrel.

10. Propiedades de texto

Propiedad	Valores	Ejemplo
`color`	Color	`color: #1f2937;`
`text-align`	`left`, `right`, `center`, `justify`	`text-align: center;`
`text-decoration`	`none`, `underline`, `overline`, `line-through`	`text-decoration: none;`
`text-transform`	`none`, `uppercase`, `lowercase`, `capitalize`	`text-transform: uppercase;`
`text-indent`	Longitud / %	`text-indent: 2em;`
`text-shadow`	`x y blur color`	`text-shadow: 1px 1px 2px #000;`
`line-height`	Número / longitud / %	`line-height: 1.6;`
`vertical-align`	`baseline`, `middle`, `top`, `bottom`, `sub`, `super`	`vertical-align: middle;`
`white-space`	`normal`, `nowrap`, `pre`, `pre-wrap`	`white-space: nowrap;`
`overflow-wrap`	`normal`, `break-word`, `anywhere`	`overflow-wrap: break-word;`

```
h2 {
  text-transform: uppercase;
  letter-spacing: 0.05em;
  text-align: center;
}

a { text-decoration: none; }
a:hover { text-decoration: underline; }

.articulo p {
  text-align: justify;
  text-indent: 2em;
  line-height: 1.7;
}
```

11. Modelo de caja

Cada elemento es una caja con cuatro capas:

```
+--------------------------- margin ---------------------------+
| +----------------------- border -------------------------+ |
| | +--------------------- padding ----------------------+ | |
| | |                  contenido                        | | |
| | +---------------------------------------------------+ | |
| +-------------------------------------------------------+ |
+-----------------------------------------------------------+
```

```
.caja {
  width: 300px;
  padding: 16px;
  border: 2px solid #0f4c81;
  margin: 24px auto;
  box-sizing: border-box;
}
```

Propiedad	Qué controla
`width` / `height`	Tamaño del contenido (o de la caja con `border-box`).
`min-width` / `max-width`	Límites de ancho.
`padding`	Espacio interno (entre contenido y borde).
`border`	Borde.
`margin`	Espacio externo.
`box-sizing`	Cómo se calcula el tamaño total.
`overflow`	Qué pasa si el contenido no entra (`visible`, `hidden`, `scroll`, `auto`).

Shorthands

```
/* Todos los lados */
margin: 10px;

/* vertical | horizontal */
margin: 10px 20px;

/* arriba | horizontal | abajo */
margin: 10px 20px 30px;

/* arriba | derecha | abajo | izquierda (sentido horario) */
margin: 10px 20px 30px 40px;

/* Lados individuales */
margin-top: 10px;
margin-right: 20px;
margin-bottom: 30px;
margin-left: 40px;
```

Igual lógica para `padding` y `border-width`.

`box-sizing`

Valor	`width` incluye...
`content-box` (por defecto)	Solo el contenido. Padding y borde **suman**.
`border-box`	Contenido + padding + borde. **Recomendado.**

```
*, *::before, *::after {
  box-sizing: border-box;
}
```

Bordes

```
.borde {
  border: 2px solid #0f4c81;       /* ancho estilo color */
  border-top: 4px dashed red;
  border-radius: 8px;              /* esquinas redondeadas */
}

.circulo {
  width: 100px;
  height: 100px;
  border-radius: 50%;
}
```

Estilos de borde: `solid`, `dashed`, `dotted`, `double`, `groove`, `ridge`, `inset`, `outset`, `none`.

`border-radius` con varios valores

Valores	Efecto
`15px`	Las cuatro esquinas iguales.
`15px 50px`	Superior-izq. e inferior-der. / superior-der. e inferior-izq.
`15px 50px 30px`	Sup-izq. / sup-der. e inf-izq. / inf-der.
`15px 50px 30px 5px`	Sup-izq. / sup-der. / inf-der. / inf-izq. (horario).

Sombras

```
box-shadow: 0 4px 12px rgba(0, 0, 0, 0.15);
/* desplazamiento-x desplazamiento-y desenfoque color */

box-shadow: inset 0 0 8px #cbd5e1;
```

12. Fondos

Propiedad	Función
`background-color`	Color de fondo.
`background-image`	Imagen de fondo (`url(...)`).

Propiedad	Función
`background-repeat`	`repeat`, `repeat-x`, `repeat-y`, `no-repeat`.
`background-position`	Posición (`left top`, `center`, `50% 50%`, etc.).
`background-attachment`	`scroll` (se mueve) o `fixed` (fijo).
`background-size`	`auto`, `cover`, `contain`, medidas.
`background`	Shorthand.

```
body {
  background-color: #f8fafc;
  background-image: url("img/fondo.png");
  background-repeat: no-repeat;
  background-position: center top;
  background-attachment: fixed;
  background-size: cover;
}

/* Shorthand */
.hero {
  background: #0f4c81 url("img/hero.jpg") no-repeat center / cover;
}
```

```
/* Múltiples fondos (CSS3) */
.multifondo {
  background-image: url("img/flores.png"), url("img/papel.png");
  background-position: right bottom, left top;
  background-repeat: no-repeat, repeat;
}
```

Degradados

```
.lineal {
  background: linear-gradient(to right, #0f4c81, #38bdf8);
}

.radial {
  background: radial-gradient(circle, #fde68a, #f59e0b);
}
```

13. `display`, `position`, `float` y `z-index`

`display`

Valor	Comportamiento
`block`	Ocupa todo el ancho, empieza en línea nueva.
`inline`	Ocupa solo su contenido, no acepta `width/height` verticales.
`inline-block`	En línea, pero acepta `width`, `height`, `margin`, `padding`.
`none`	No se muestra ni ocupa lugar.
`flex`	Contenedor Flexbox.
`grid`	Contenedor Grid.

```
.oculto { display: none; }        /* desaparece del layout */
.invisible { visibility: hidden; } /* ocupa lugar, pero no se ve */
```

`position`

Valor	Referencia	Comportamiento
`static`	—	Flujo normal (por defecto).
`relative`	Su posición original	Se desplaza sin salir del flujo.
`absolute`	Ancestro posicionado más cercano	Sale del flujo.
`fixed`	Viewport	Queda fijo al hacer scroll.
`sticky`	Scroll + contenedor	Se “pega” al llegar a un punto.

```
.contenedor { position: relative; }

.badge {
  position: absolute;
  top: 8px;
  right: 8px;
}

.barra {
  position: fixed;
  top: 0;
  left: 0;
  width: 100%;
}

.encabezado {
  position: sticky;
  top: 0;
}
```

Propiedades de desplazamiento: `top`, `right`, `bottom`, `left`, `z-index`.

`float` y `clear`

```
.imagen {
  float: left;
  margin-right: 1rem;
}

.limpiar {
  clear: both;   /* left | right | both */
}
```

> `float` se usó históricamente para layout; hoy se usa **Flexbox/Grid**. `float` queda para texto que rodea una imagen.

`z-index`

```
.modal {
  position: fixed;
  z-index: 1000;
}
```

- Solo funciona en elementos **posicionados** (o con `flex` / `grid` en ciertos casos).
- Mayor número = más adelante.
- Crea "contextos de apilamiento"; un hijo no puede superar el `z-index` de su padre.

---

14. Flexbox

```
.contenedor {
  display: flex;
  flex-direction: row;        /* row | row-reverse | column | column-reverse */
  flex-wrap: wrap;            /* nowrap | wrap | wrap-reverse */
  justify-content: space-between; /* eje principal */
  align-items: center;        /* eje transversal */
  gap: 1rem;
}
```

Propiedad (contenedor)	Valores
`flex-direction`	`row`, `row-reverse`, `column`, `column-reverse`.
`flex-wrap`	`nowrap`, `wrap`, `wrap-reverse`.
`justify-content`	`flex-start`, `flex-end`, `center`, `space-between`, `space-around`, `space-evenly`.
`align-items`	`stretch`, `flex-start`, `flex-end`, `center`, `baseline`.
`align-content`	Para varias líneas.
`gap`	Espacio entre ítems.

Propiedad (ítem)	Valores
`flex-grow`	Cuánto crece (0 por defecto).

Propiedad (ítem)	Valores
`flex-shrink`	Cuánto se encoge (1 por defecto).
`flex-basis`	Tamaño base (`auto`).
`flex`	Shorthand: `flex: 1 1 200px` .
`order`	Orden visual.
`align-self`	Alineación individual.

```
/* Barra de navegación */
.nav {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 1rem;
}

/* Tarjetas que se reparten el ancho */
.cards {
  display: flex;
  flex-wrap: wrap;
  gap: 1rem;
}

.card {
  flex: 1 1 240px;   /* crece, se encoge, base 240px */
}
```

> **Eje principal** = `flex-direction` ; `justify-content` lo recorre. **Eje transversal** = perpendicular; `align-items` lo recorre.

15. Grid

```
.grid {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 1rem;
}
```

Propiedad (contenedor)	Función
`grid-template-columns` / `rows`	Define las pistas (columnas/filas).
`grid-template-areas`	Layout con nombres.
`gap` (`row-gap` , `column-gap`)	Espacio entre celdas.
`justify-items` / `align-items`	Alineación dentro de cada celda.
`justify-content` / `align-content`	Alineación de la grilla completa.
`grid-auto-flow`	Cómo se acomodan los ítems automáticos.

Propiedad (ítem)	Función
`grid-column`	`1 / 3` , `span 2` .
`grid-row`	`1 / 2` , `span 3` .
`grid-area`	Nombre o `fila / columna / fila-fin / columna-fin` .
`justify-self` / `align-self`	Alineación individual.

```
.layout {
  display: grid;
  grid-template-columns: 240px 1fr;
  grid-template-rows: auto 1fr auto;
  grid-template-areas:
    "header header"
    "sidebar main"
    "footer footer";
  gap: 1rem;
  min-height: 100vh;
}
```

```
.layout > header { grid-area: header; }
.layout > aside  { grid-area: sidebar; }
.layout > main   { grid-area: main; }
.layout > footer { grid-area: footer; }
```

```
/* Responsive sin media query */
.auto-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(240px, 1fr));
  gap: 1rem;
}
```

Flexbox	Grid
Una dimensión (fila **o** columna).	Dos dimensiones (filas **y** columnas).
Basado en el contenido.	Basado en la grilla.
Ideal para barras y grupos.	Ideal para layouts completos.

16. Responsive: media queries

```
/* Mobile first: estilos base para pantallas chicas */
body { font-size: 16px; }

@media (min-width: 768px) {
  body { font-size: 17px; }
  .layout { grid-template-columns: 240px 1fr; }
}

@media (min-width: 1024px) {
  .layout { grid-template-columns: 280px 1fr; }
}
```

Operador	Ejemplo
`min-width`	`@media (min-width: 768px)`
`max-width`	`@media (max-width: 767px)`
Rango	`@media (width >= 600px) and (width < 900px)`
Orientación	`@media (orientation: landscape)`
Preferencia	`@media (prefers-color-scheme: dark)`
Impresión	`@media print`

```
@media print {
  nav, footer, .no-print { display: none; }
  body { font-size: 12pt; color: black; }
  a::after { content: " (" attr(href) ")"; }
}
```

Breakpoints habituales: 576 px, 768 px, 992 px, 1200 px. No son obligatorios: se eligen según el contenido.

```
<meta name="viewport" content="width=device-width, initial-scale=1">
```

> Sin la meta `viewport`, el celular simula un ancho de escritorio y las media queries no funcionan como se espera.

17. Transiciones, transformaciones y animaciones

Transiciones

```
.boton {
  background: #0f4c81;
  transition: background-color 0.3s ease, transform 0.2s ease;
}

.boton:hover {
  background: #1d4ed8;
  transform: translateY(-2px);
}
```

```
transition: propiedad duración timing-function retardo;
transition: all 0.3s ease 0s;
```

Timing function	Efecto
`linear`	Velocidad constante.
`ease`	Suave (por defecto).
`ease-in`	Empieza lento.
`ease-out`	Termina lento.
`ease-in-out`	Lento en ambos extremos.
`cubic-bezier(...)`	Personalizada.

Transformaciones

```
.transformar:hover {
  transform: translate(10px, 0) rotate(5deg) scale(1.1);
}

.transformar {
  transform-origin: center;
}
```

Función	Efecto
`translate(x, y)`	Mueve.
`translateX` / `translateY`	Mueve en un eje.
`scale(n)`	Escala.
`rotate(deg)`	Rota.
`skew(deg)`	Inclina.
`matrix(...)`	Transformación completa.

> `transform` no modifica el flujo del documento: el espacio original se conserva.

Animaciones

```
@keyframes aparecer {
  from { opacity: 0; transform: translateY(10px); }
  to   { opacity: 1; transform: translateY(0); }
}

.tarjeta {
  animation: aparecer 0.4s ease-out forwards;
}
```

```
animation: nombre duración timing delay iteraciones dirección fill-mode;
animation: aparecer 0.4s ease-out 0s 1 normal forwards;
```

Propiedad	Valores
`animation-iteration-count`	`1`, `infinite`, número.
`animation-direction`	`normal`, `reverse`, `alternate`.
`animation-fill-mode`	`none`, `forwards`, `backwards`, `both`.
`animation-play-state`	`running`, `paused`.

18. Extras de CSS3

Esquinas redondeadas

```
#rcorners {
  border-radius: 25px;
  background: #8ac007;
  padding: 20px;
  width: 200px;
```

```
  height: 150px;
}
```

Bordes con imagen

```
#borderimg {
  border: 10px solid transparent;
  padding: 15px;
  border-image: url("img/border.png") 30 round;
  /* round | stretch | repeat */
}
```

Múltiples fondos

```
#ejemplo {
  background-image: url("img/flores.png"), url("img/papel.png");
  background-position: right bottom, left top;
  background-repeat: no-repeat, repeat;
}
```

Otros módulos de CSS3

Módulo	Para qué
CSS Color	Colores, `rgba`, `hsl`.
CSS Selectors	Selectores avanzados.
CSS Media Queries	Diseño responsive.
CSS Backgrounds & Borders	Fondos, bordes, sombras.
CSS 2D/3D Transformations	`transform`.
CSS Transitions / Animations	Movimiento.
CSS Flexible Box Layout	Flexbox.
CSS Grid Layout	Grillas.
CSS Fonts	`@font-face`, fuentes.

---

19. Errores típicos y checklist

Error	Causa	Solución
El estilo no se aplica	Selector mal escrito o ruta incorrecta.	Revisar selector y `href`; abrir DevTools.
"Gana" una regla que no esperaba	Especificidad o `!important`.	Inspeccionar la cascada en DevTools.
El `padding` agranda la caja	`box-sizing: content-box`.	`box-sizing: border-box`.
Márgenes que se "colapsan"	Márgenes verticales entre bloques.	Usar `padding`, `gap` o `overflow: auto`.
`z-index` no funciona	El elemento no está posicionado.	Agregar `position: relative/absolute/fixed`.
`absolute` se sale del contenedor	El padre no está posicionado.	`position: relative` en el contenedor.
El layout se rompe en mobile	Falta `viewport` o media queries.	Agregar meta viewport y `@media`.
`height: 100%` no funciona	El padre no tiene altura definida.	Usar `100vh` o `min-height`.
El texto no se centra verticalmente	Se usó `vertical-align` en un bloque.	Flexbox: `align-items: center`.
`::before` no aparece	Falta `content`.	Agregar `content: ""`.
La fuente web no carga	Ruta o formato incorrecto, o CORS.	Revisar `src`, `format()` y `font-display`.
La animación no se repite	Falta `animation-iteration-count`.	`infinite`.

Checklist antes de entregar

- [ ] CSS externo enlazado con `<link>`.
- [ ] `box-sizing: border-box` global.
- [ ] Variables para colores y espaciados.
- [ ] Selectores cortos y por función (`.alerta`, no `.rojo`).
- [ ] Sin `!important` innecesarios.
- [ ] Sin estilos inline salvo excepciones.
- [ ] Tipografía con familia de respaldo.

- [ ] Contraste y foco visibles (accesibilidad).
- [ ] Layout probado en móvil, tablet y escritorio.
- [ ] Meta `viewport` presente.
- [ ] Sin desplazamiento horizontal en mobile.
- [ ] `:hover` acompañado de `:focus-visible`.

---

20. Plantilla completa

```
/* ===== 1. Reset y variables ===== */
*, *::before, *::after {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

:root {
  --color-primario: #0f4c81;
  --color-acento: #0ea5e9;
  --color-texto: #1f2937;
  --color-fondo: #f8fafc;
  --color-borde: #cbd5e1;
  --radio: 8px;
  --sombra: 0 4px 12px rgba(0, 0, 0, 0.08);
  --espacio: 1rem;
  --ancho-max: 1100px;
}

/* ===== 2. Base ===== */
html { font-size: 16px; }

body {
  font-family: "Segoe UI", Arial, Helvetica, sans-serif;
  font-size: 1rem;
  line-height: 1.6;
  color: var(--color-texto);
  background: var(--color-fondo);
}

img { max-width: 100%; height: auto; display: block; }

a { color: var(--color-primario); text-decoration: none; }
a:hover { text-decoration: underline; }

h1, h2, h3 { line-height: 1.2; color: var(--color-primario); }
h1 { font-size: 2.25rem; }
h2 { font-size: 1.5rem; margin-bottom: 0.5rem; }

/* ===== 3. Layout ===== */
.contenedor {
  width: min(100% - 2rem, var(--ancho-max));
  margin-inline: auto;
}

.layout {
  display: grid;
  grid-template-columns: 1fr;
  gap: var(--espacio);
  padding-block: 2rem;
}

@media (min-width: 768px) {
  .layout { grid-template-columns: 240px 1fr; }
}

/* ===== 4. Componentes ===== */
.card {
  background: #fff;
  border: 1px solid var(--color-borde);
  border-radius: var(--radio);
  box-shadow: var(--sombra);
  padding: var(--espacio);
  transition: transform 0.2s ease, box-shadow 0.2s ease;
}

.card:hover {
```

```
  transform: translateY(-3px);
  box-shadow: 0 8px 20px rgba(0, 0, 0, 0.12);
}

.boton {
  display: inline-block;
  padding: 0.6rem 1.2rem;
  border: none;
  border-radius: var(--radio);
  background: var(--color-primario);
  color: #fff;
  font: inherit;
  cursor: pointer;
  transition: background 0.2s ease;
}

.boton:hover { background: var(--color-acento); }

/* ===== 5. Utilidades ===== */
.oculto { display: none; }
.texto-centro { text-align: center; }

@media print {
  .no-print { display: none; }
  body { background: #fff; color: #000; font-size: 12pt; }
}
```

Fin del cheatsheet.
Si tenés que priorizar para el parcial: sintaxis y formas de inclusión, selectores y combinadores, pseudo-clases/elementos, cascada y especificidad, `box-sizing` y modelo de caja, `display` / `position`, Flexbox, Grid, media queries y `@font-face`.